

Natív Programozás ZH

SZTE Szoftverfejlesztés Tanszék

2025. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
 - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot g++ fordítóval és a
-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1
paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
 - Futási hiba 6: Memória- vagy időkorlát túllépés.
 - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
 - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlinde克斯, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen .cpp forrásfájl lehet feltölteni (zip fájl nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.

Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy "3 alma", akkor az elvárt kimenet idézőjelek nélkül az 3 alma, de a szóköz szükséges!
 - A tesztesetekben nem lesz ékezetes szöveg kiírása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
 - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- A ZH során használható segédanyag elérhető bíróban.
 - <https://biro.inf.u-szeged.hu/kozos/native/>

Dinamikus memória követése

Ebben a feladatban lehetőség van egy speciális konstrukció használatára ami segít az allokált memória követésére. Amennyiben a `new` után egy `TRACK_INFO`-t írunk a biro rendszere ki tudja írni, hogy hol került allokálásra az adott memória terület ami nem lett felszabadítva (bizonyos tesztek esetén). Ezt az információt a teszt eset `.stdout` kiterjesztésű fájljába írja bele.

Példa használat: `new TRACK_INFO Engine(...)`

Példa kimenet az `stdout` fájlban:

```
[ FAIL ] Memory error occured. Here is the map of the allocated memory addresses:
---| FAIL | 0xd7ee40 is not freed. filename: feladat.cpp:123 in function: <function>
---| FAIL | 0xd7eeb0 is not freed. filename: feladat.cpp:123 in function: <function>
---| OK   | 0xd7ef20 is nicely freed.
```

Figyelem! Amennyiben nem biro-n kerül fordításra a program, ilyen memória allokáció követés nem történik.

Kiindulás

A kiindulási feladatban kapott osztály módosítása vagy bővítése szükséges lehet!

A kiindulási osztály, `Part`, egy repülőgép alkatrészt reprezentál.

- Egy alkatrész lehet aktív vagy inaktív. Minden alkatrész alapból inaktív.
- Az alkatrészeknek lehet azonosítójuk illetve lehetséges, hogy használat során degradálódnak.

Part - javítás

Az alkatrész legyen absztrakt! Tipp: a fordítási hibás metódus(ok) átalakításával meg is tehető.

Engine - osztály

Készítsd el az `Engine` osztályt, mely a hajtóművet reprezentálja. A hajtómű egy speciális alkatrész. Legyen az `Engine` példányosítható, azaz ne legyen absztrakt!

- A hajtóműnek van egy teljesítmény adattagja (típusa előjeltelen egész).
- Létrehozáskor a teljesítmény és ID az elvárt paraméter (ebben a sorrendben).

identify

Valósítsd meg, hogy az `identify` metódus a következő formában adja vissza az értéket:

`Engine: <power> <identity>`

Ahol a `<power>` a hajtómű teljesítménye, az `<identity>` pedig az `Ős` osztály `identify` metódus értéke. Minden részt EGY szóköz választ el! A metódus polimorfikusan tudjon működni ha `Part`-ként hivatkozunk rá!

LifeSupport - osztály

Készítsd el a LifeSupport osztályt, mely a létfontartásért felelős speciális **alkatrész**!

- Az osztály példányosításakor meg kell adni, hogy milyen maximum magasságig tud működni és, hogy milyen hosszan képes megfelelően működni (előjel nélküli egészek, a konstruktor ebben a sorrendben várja őket).
- Minden LifeSupport ID-ja: "LS".
- Az osztályból csakis aktív alkatrész készülhet!

operator!

Valósítsd meg, hogy a ! operátor meghívásával ne lehessen inaktívvá változtatni egy LifeSupport objektumot. Ügyelj rá, hogy tudjon polimorfikusan viselkedni.

identify

A metódus a következő formátumban adjon vissza egy std::string-et:

LifeSupport: [<duration>@<altitude>]

Ahol a <duration> a megadott hossz, ameddig működni képes az alkatrész. Az <altitude> pedig a legnagyobb magasság amíg megfelelően működik az alkatrész.

degrade

Mikor egy LifeSupport alkatrész degradálódik, akkor eggyel csökken a hossz érték, amíg működni képes. **Nem kell felkészülni 0-ra vagy az alá csökkenő esetre!**

Airplane - osztály

Az Airplane osztály egy repülőgépet ábrázol. A repülőgép különféle alkatrészekkel szerelhető fel, melyeket beszerelés sorrendjében tartanak számon.

- A repülőgép az alkatrészeket polimorfikusan tárolja és felelős a memória megfelelő kezeléséért (a repülőgép birtokolja a pointereket).
- Egy repülőgép létrehozásakor meg kell adni, hogy mi a tanácsolt hajtómű (Engine) szám. Alapból nincs hajtómű a repülőgépen.

addPart

Készítsd el az addPart függvényt, mely egy alkatrész vár (referenciaként) és beszereli azt a repülőgépbe. Ha egy hajtóművet (Engine) kap, akkor azt csak akkor szereljük be, ha nem érte még el az ajánlott hajtóműszámot. **Fontos, hogy a repülő ne a külső alkatrészsre hivatkozzon!**

Ez a metódus biztosan kelleni fog a többi metódus teszteléséhez is!

checklist

A `checklist` metódus leellenőrzi a repülőgép adatait. Egy `std::string`-et ad vissza, ami az alábbi módon van felépítve:

```
Airplane:
<provided> out of <engines>
<identity1>
...
<identityN>
```

Ahol minden sor egy sortörés karakterrel végződik. Az "Airplane:" szöveg egy kötelező fix sor. A `<provided>` helyére a felszerelt hajtóművek számát kell írni. Az `<engines>` helyére az ajánlott hajtómű számot kell írni. A további sorokba az **aktív** alkatrészek `identify` kimenetét kell írni. Minden alkatrész után található sortörés! Egy példa:

```
Airplane:
0 out of 4
LifeSupport: [4@1500]
LifeSupport: [1@3500]
```

Ez a metódus biztosan kelleni fog a többi metódus teszteléséhez is!

operator!

Valósítsd meg a `!` operátort! Az operátor minden alkatrésze meghívja a `!` operátort.

inventory

Valósítsd meg az `inventory` metódust! A metódus feladata, hogy egy lista alapján azonosítsa be a hiányzó alkatrészeket.

- A metódus egy `std::string`-eket tároló `std::vector`-t vár paraméterül, melyben a keresett alkatrészek `identify` értékei vannak.
- Nem számít, hogy aktív vagy inaktív egy alkatrész.
- A metódus összegyűjti, hogy mely alkatrészek **nem** találhatók meg a repülőgépen és azokat (a hiányzó alkatrészeket reprezentáló sztringeket) egy, a paraméterhez hasonló, `std::string` vektorban visszaadja.
- Az eredményben az alkatrészek ugyanabban a sorrendben szerepeljenek mint a paraméterben!

Memóriakezelés

A repülőgép gondoskodjon megszűnésekor az alkatrészek felszabadításáról. Legyen az osztály megfelelő módon másolható (copy konstruktor, értékadás operátor).